CS 412 / 512
Introduction to Computer Graphics

Zhu Wang
Assistant Professor of Computer Science



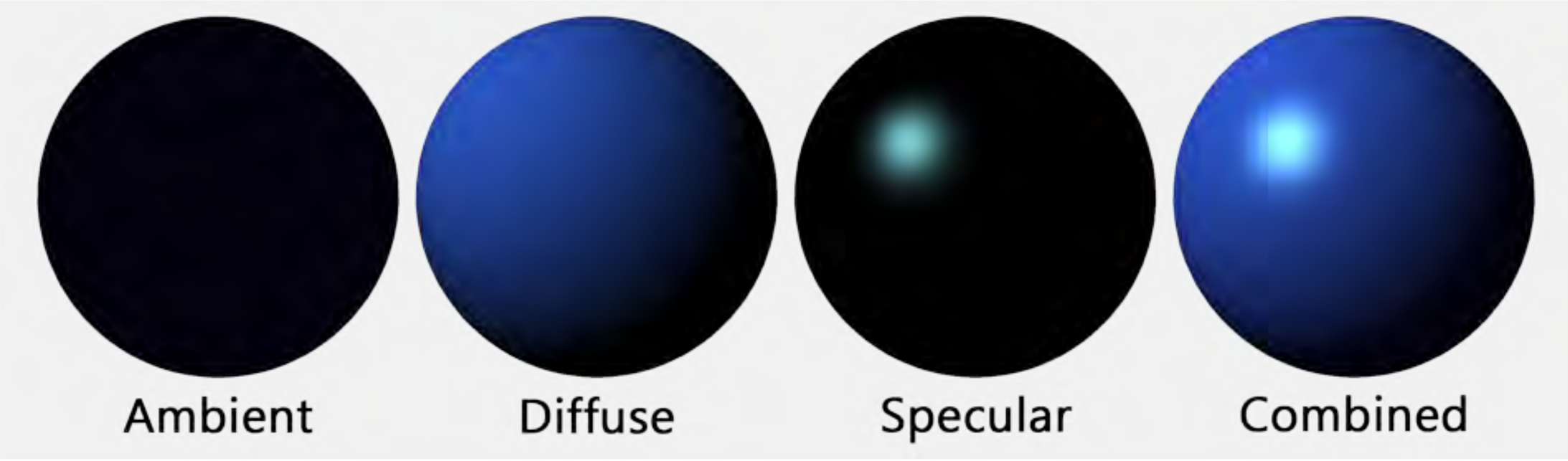
COMPUTER SCIENCE

Lighting & Shading

Lighting

Ambient, diffuse, specular

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$



[source:clara.io]

Ambient Reflection

K_a Ambient reflection coefficient

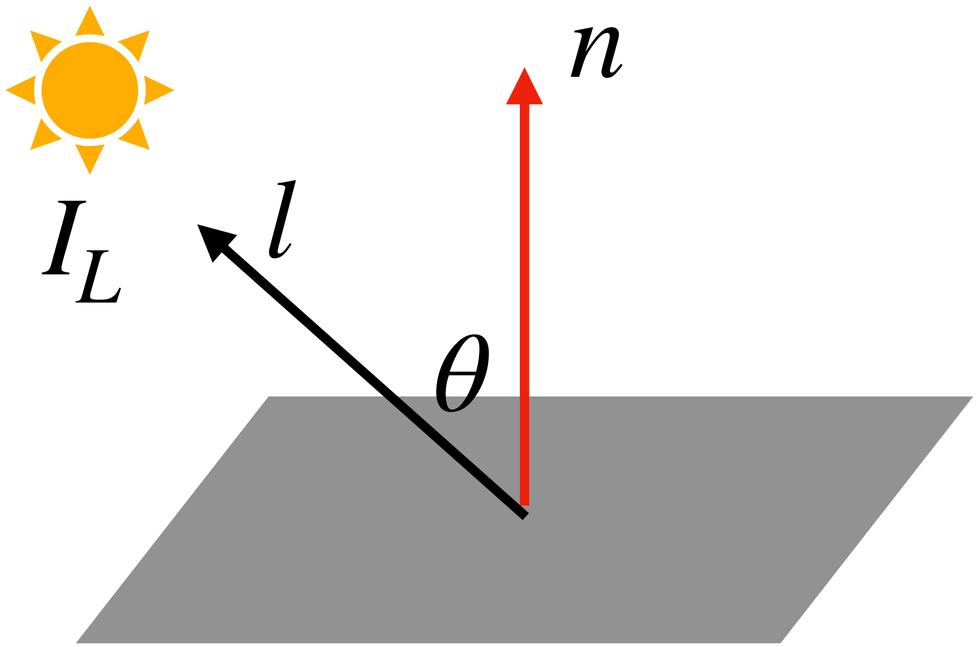
I_L Incoming light intensity

$$I_a = K_a I_L$$

Ambient doesn’t depend on the normal, the light direction, or the view direction

Diffuse reflection

Lambertian model



K_d diffuse reflection coefficient

I_L Incoming light intensity

$$I_d = I_L K_d max(0, cos\theta)$$

Specular Reflection

Reflected light bounces off at the same angles as the incident ray



Diffuse

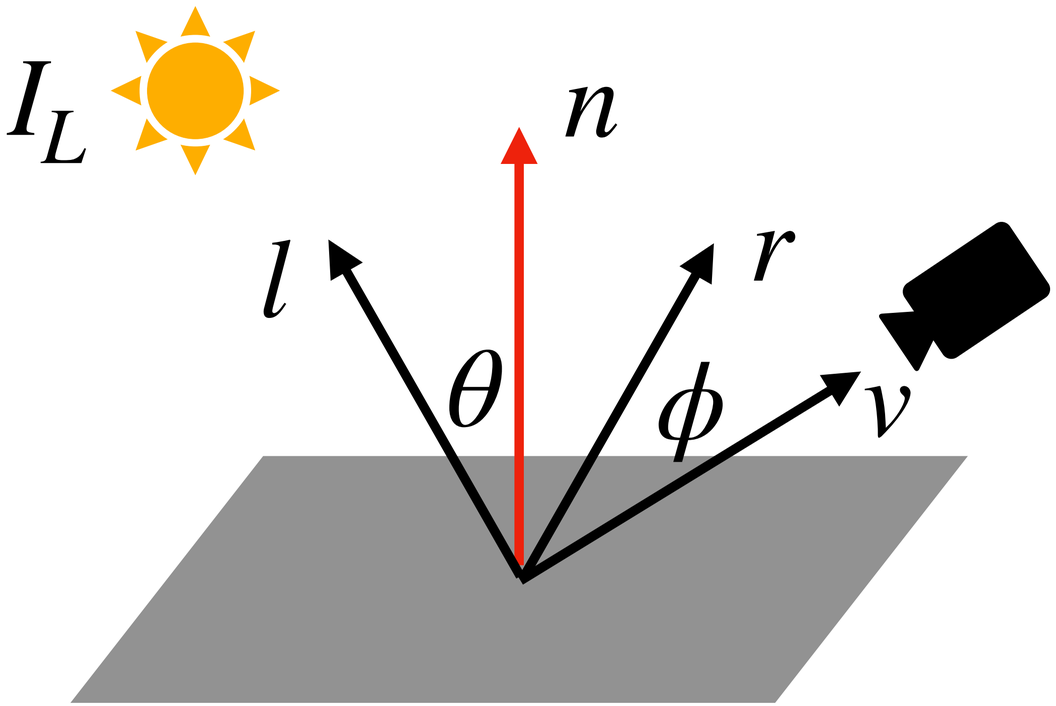


[source:Wikipedia]

Specular

Specular reflection

Phong model



K_s Specular reflection coefficient

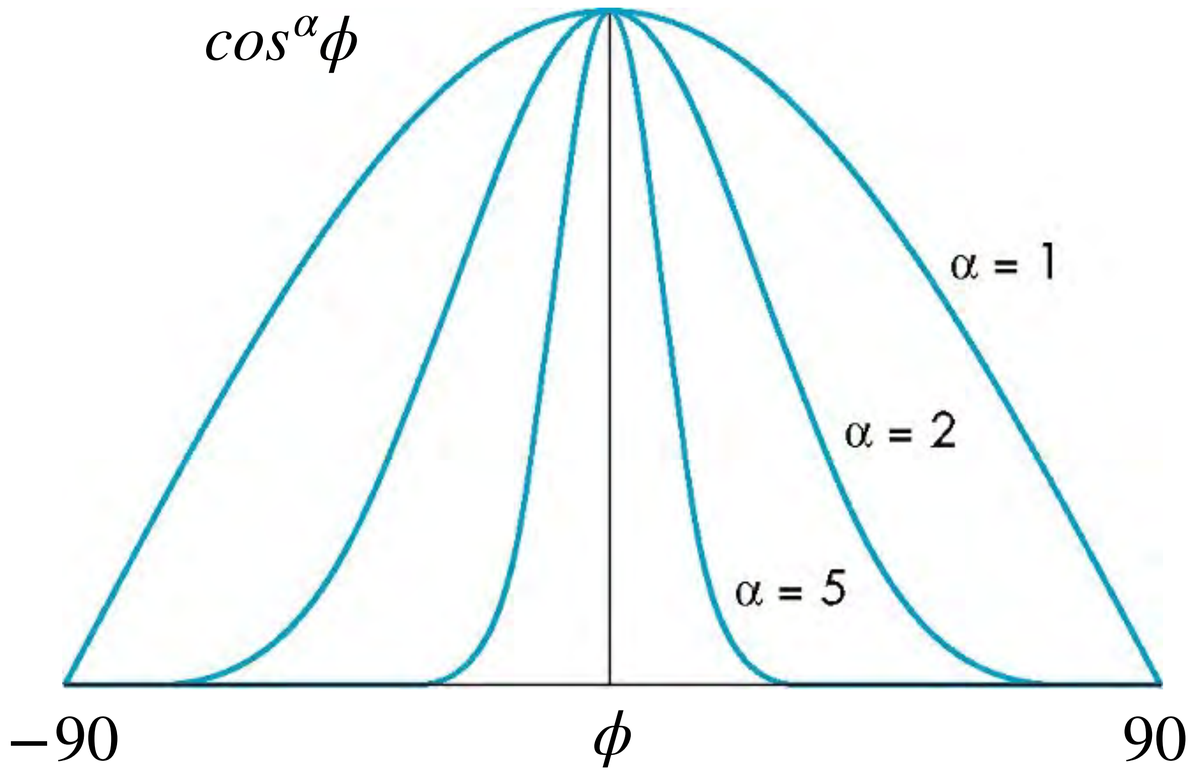
I_L Incoming light intensity

α Shininess coefficient(specular exponent)

ϕ Angle between r and v

$$I_s = I_L K_s (max(0, cos\phi))^{\alpha}$$

Specular Exponent



$\alpha \in [100, 200] \longrightarrow$ metal like

$\alpha \in [5, 10] \longrightarrow$ plastic like



$K_a = 0, K_d = 0.5, K_s = 0$



$K_a = 0.5, K_d = 0.5, K_s = 0$



$K_a = 0.6,\ K_d = 0.1,\ K_s = 0.6,\ \alpha = 10$



$K_a = 0.6,\ K_d = 0.6,\ K_s = 0.6,\ \alpha = 10$

Specular



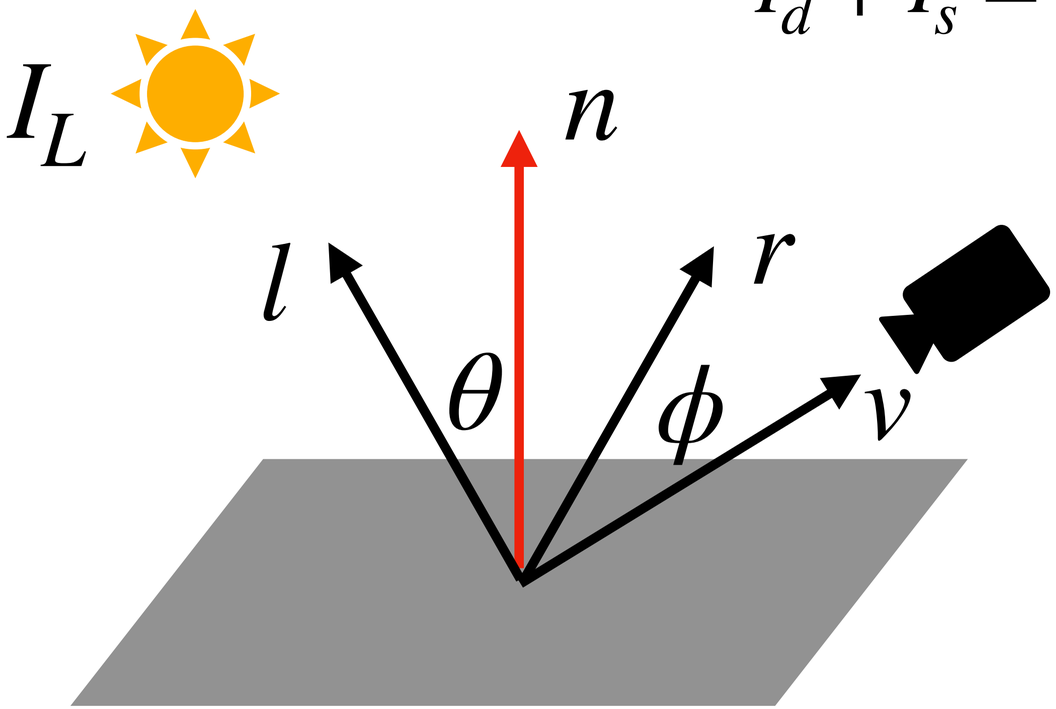
$K_a = 0.6,\ K_d = 0.2,\ K_s = 0.8,\ \alpha = 1$



$K_a = 0.6,\ K_d = 0.2,\ K_s = 0.8,\ \alpha = 100$

Phong model

$$I_d + I_s = I_L(K_d max(0, cos\theta) + K_s(max(0, cos\phi))^{\alpha})$$

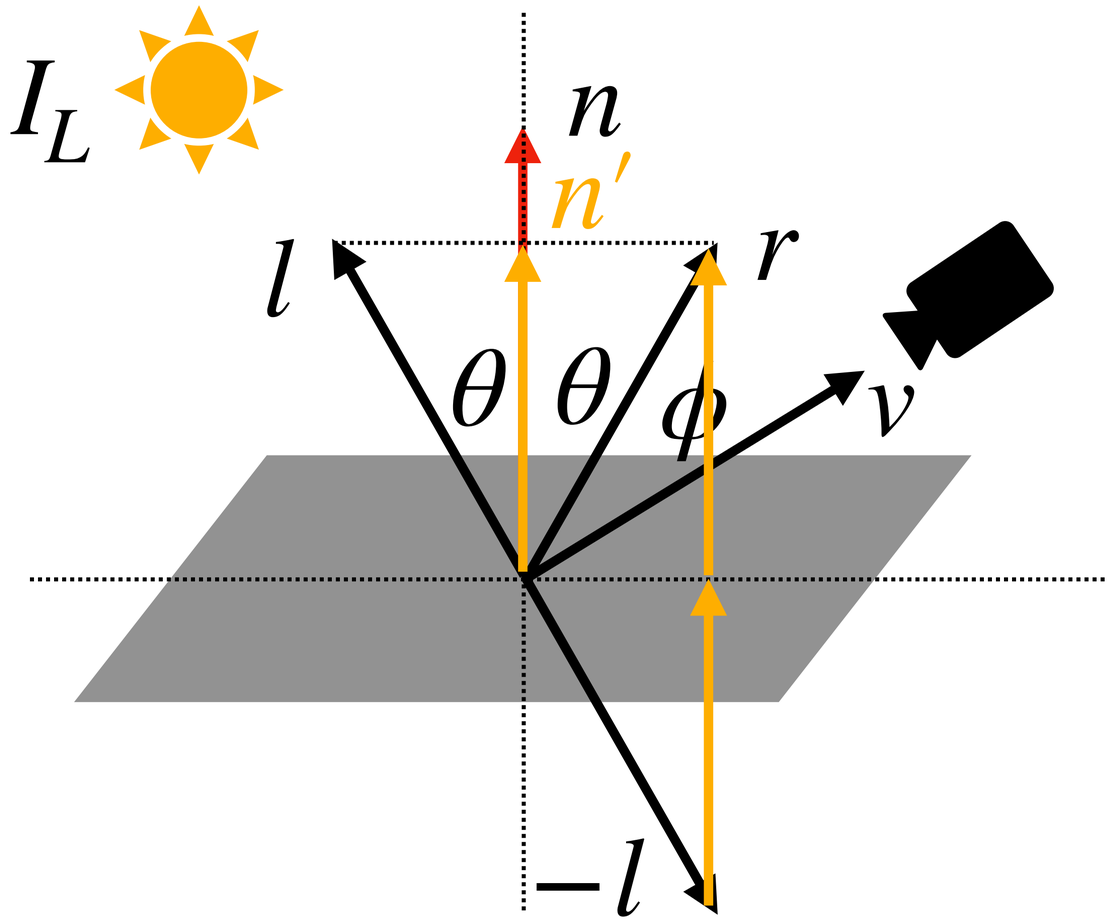


$$cos\theta = \frac{n \cdot l}{\|n\| \cdot \|l\|}$$

$$cos\phi = \frac{n \cdot r}{\|n\| \cdot \|r\|}$$

Phong model

How to get r

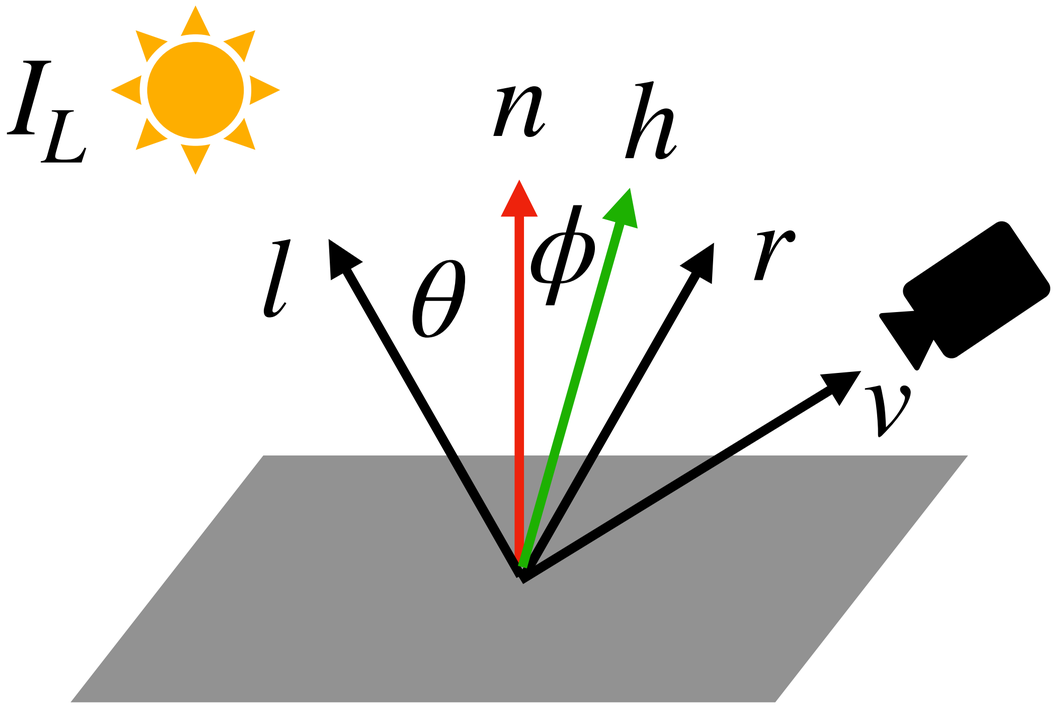


$$n' = (\|l\| \cdot cos\theta)n$$
$$= (l \cdot n)n$$

$$r = 2n' - l$$
$$= 2(l \cdot n)n - l$$

Blinn-Phong Model

Halfway vector h



$$h = \frac{l + v}{\|l + v\|}$$

$$I_s = I_L K_s (max(0, n \cdot h))^{\alpha}$$

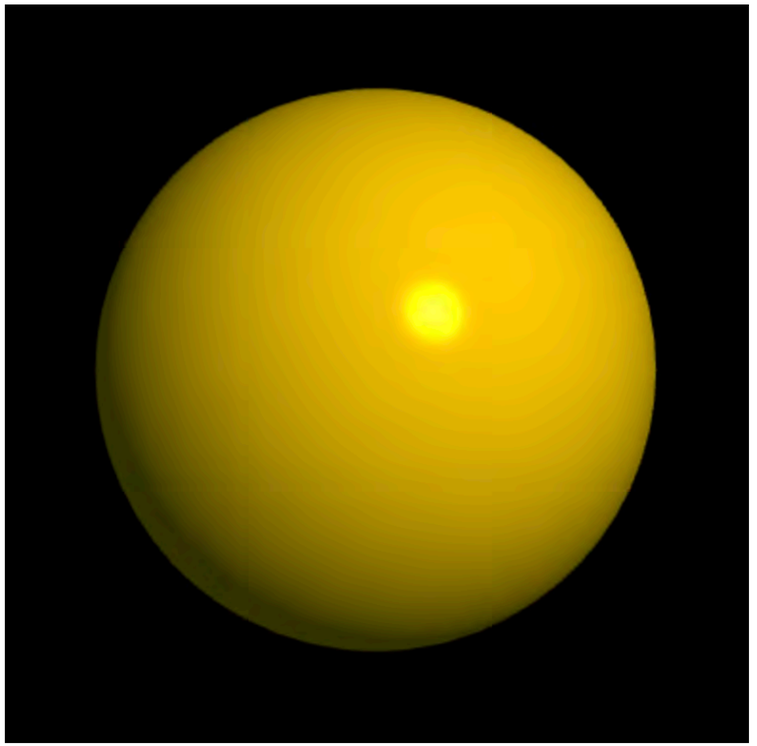
Adding Color to Shading Models

K_a, K_d, K_s as colors

K_a Ambient color, base tint

K_d Diffuse color, main color of the material

K_s Specular color, highlight color



$K_a = (0.2, 0.2, 0)$

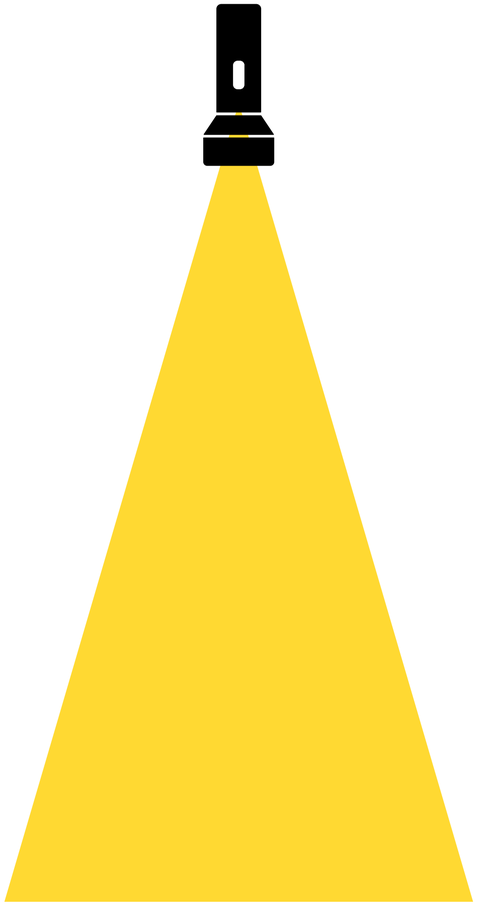
$K_d = (0.8, 0.6, 0)$

$K_s = (1, 0.9, 0.3), \alpha = 100$

Light Sources

Point light

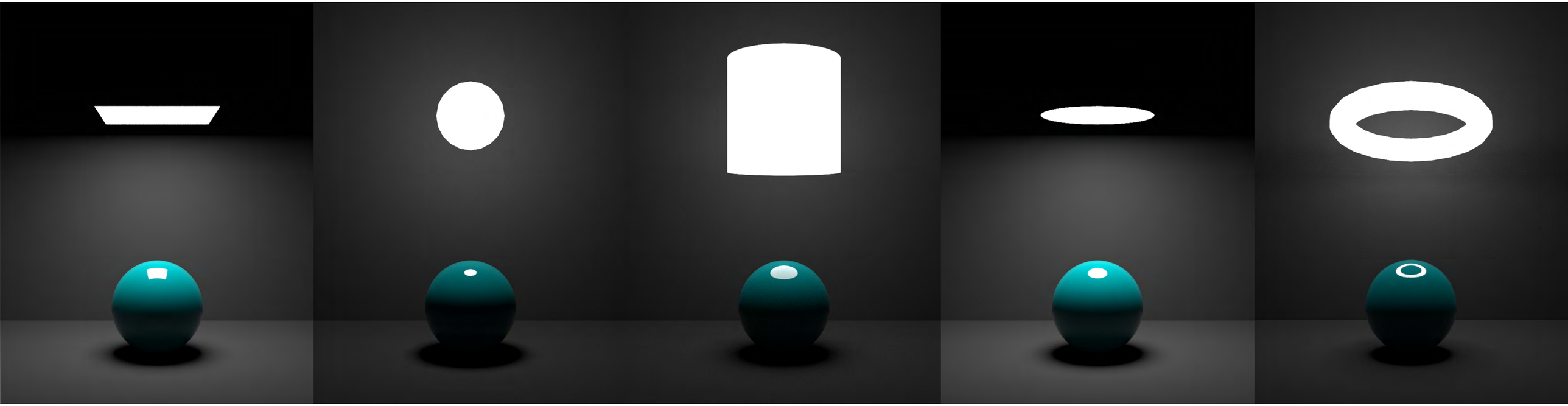
Directional light



Spot light

Light Sources

Area light



[source: AMD Radeon]

Light Sources

Multiple light sources in a scene







Multiple Light Sources

Adding contributions from all sources

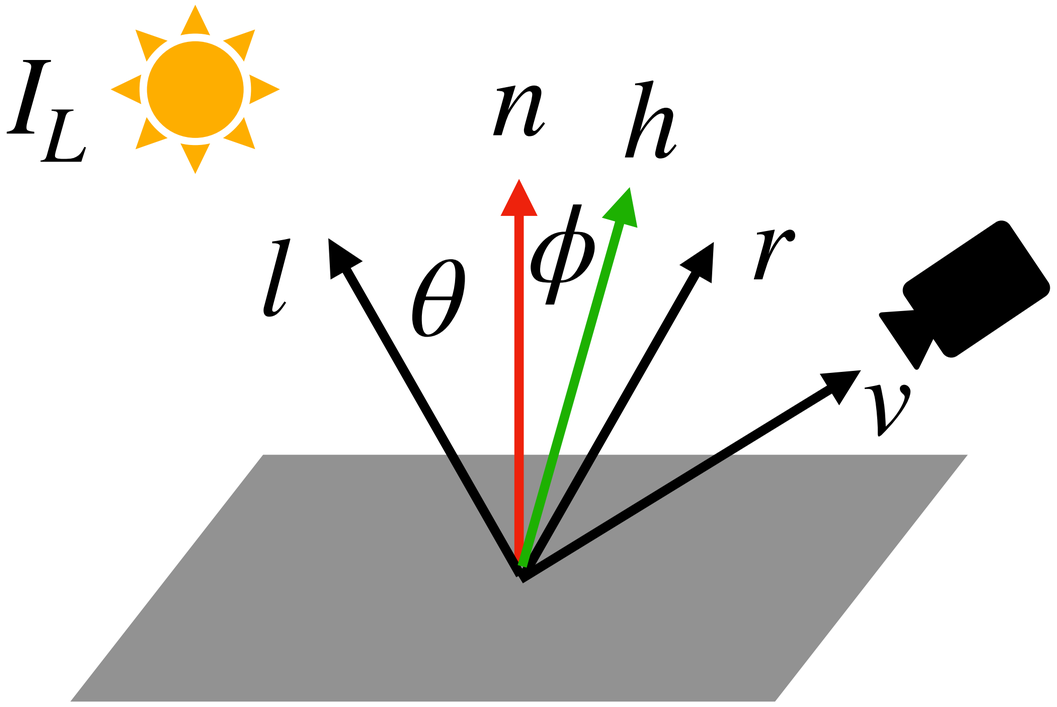
For each light source and each surface:

$$I = I_a K_a + I_d K_d (l \cdot n) + I_s K_s (v \cdot r)^{\alpha}$$

For n light sources and each surface:

$$I_{final} = I_a K_a + \sum_{i=1}^{n} [I_{d,i} K_d (l_i \cdot n) + I_{s,i} K_s (v \cdot r_i)^{\alpha}]$$

Computing Lighting & Shading



What do we need?

Lights

Normals

Material properties

Compute in application or in shaders

Point Light

Color components and position

```
let ambient = vec4(1.0, 0.0, 0.0, 1.0);
let diffuse = vec4(1.0, 0.0, 0.0, 1.0);
let specular = vec4(1.0, 0.0, 0.0, 1.0);
let light_pos = vec4(1.0, 2.0, 3.0, 1.0);
```

vec3 is sufficient for ambient, diffuse and specular in most cases.

We can use vec4 for RGBA if we want to consider transparency.

The position is in homogeneous coordinates

- w =1.0. a finite location
- w =0.0 a parallel source with the given direction

Light Attenuation

d - distance from the light to the point on the surface

$$I \propto \frac{1}{d^2}$$

Constant term to avoid infinite brightness at the origin

$$\frac{1}{1 + d^2}$$

Light Attenuation

Flexible quadratic equation

$$f_{att} = \frac{1}{a + bd + cd^2}$$

$$I = I_a K_a + f_{att}[I_d K_d (l \cdot n) + I_s K_s (v \cdot r)^{\alpha}]$$

Attenuation is usually applied to diffuse and specular

d - distance from the light to the point on the surface

a, b, c are attenuation coefficients that control how fast the light fades:

- a - constant term controls the base brightness
- b - linear rate of fading
- c - quadratic rate of fading, physically realistic falloff

Light Attenuation

$$f_{att} = \frac{1}{a + bd + cd^2}$$

Examples

a	b	c	f_{att}	
1.0	0.0	0.0	1.0	No attenuation
0.5	0.1	0.0	$f_{att} = \frac{1}{0.5 + 0.1d}$	Slowly decreasing
0.0	0.0	2.0	$f_{att} = \frac{1}{2d^2}$	Fast falloff, localized light
1.0	0.09	0.032	$f_{att} = \frac{1}{1 + 0.09d + 0.032d^2}$	Smooth natural fade (practical empirical default)

Spot Light

Directional control and falloff

```
let ambient = vec4(1.0, 0.0, 0.0, 1.0);
let diffuse = vec4(1.0, 0.0, 0.0, 1.0);
let specular = vec4(1.0, 0.0, 0.0, 1.0);
let light_pos = vec4(1.0, 2.0, 3.0, 1.0);
```

vec3 is sufficient for ambient, diffuse and specular in most cases.

We can use vec4 for RGBA if we want to consider transparency.

The position is in homogeneous coordinates

- w =1.0. a finite location
- w =0.0 a parallel source with the given direction

Computing Lighting & Shading

Apply transformations and normalization to normals in vertex shader:

```
vNormal = normalize(uNormalMatrix * aNormal)
```

Apply lighting effects per pixel in fragment shader:

```
  // Ambient (only computed once, not per light)
  vec3 ambient = uKa * vColor;

  vec3 viewDir = normalize(uViewPos - vPosition);

  // Calculate contribution from both lights
  vec3 light1 = calculateLight(uLightPos1, vNormal, viewDir);
  vec3 light2 = calculateLight(uLightPos2, vNormal, viewDir);

  vec3 result = ambient + light1 + light2;
  fragColor = vec4(result, 1.0);
```

Computing Diffuse and Specular

```
vec3 calculateLight(vec3 lightPos, vec3 normal, vec3 viewDir)
{ // Diffuse
  vec3 lightDir = normalize(lightPos - vPosition);
  float cosTheta = max(dot(normal, lightDir), 0.0);
  vec3 diffuse = uKd * cosTheta * vColor;
  // Specular
  vec3 reflectDir = -reflect(lightDir, normal);
  float spec = pow(max(dot(viewDir, reflectDir), 0.0), uAlpha);
  vec3 specular = uKs * spec * vec3(1.0);
  return diffuse + specular;
}
```

reflect function is a built-in function, it is I - 2.0 * dot(N, I) * N